7. Construct a C Program to Implement Non-Preemptive SJF Algorithm

Code - Part 1 (Lines 1–44)

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int at[10], bt[10], pr[10];
6      int n, i, j, temp, time = 0, count, over = 0;
7      int sum_wait = 0, sum_turnaround = 0, start;
8      float avgwait, avgturn;
9
10     printf("Enter the number of processes:\n");
11     scanf("%d", &n);
12
13     for(i = 0; i < n; i++)
14     {
15         printf("Process %d:\n", i + 1);
16         printf("Enter arrival time and burst time:\n");
17         scanf("%d %d", &at[i], &bt[i]);
18         pr[i] = i + 1;
19     }
20
21     // Sort based on arrival time
22     for(i = 0; i < n - 1; i++)
23     {
24         for(j = i + 1; j < n; j++)
25         {
26             if(at[i] > at[j])
27             {
28                 temp = at[i]; at[i] = at[j]; at[j] = temp;
29                 temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
30                 temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
31             }
32         }
33     }
34
35     printf("\nProcess\tAT\tBT\tStart\tEnd\tWT\tTAT\n");
36
37     while(over < n)
38     {
39         count = 0;
40
41         for(i = over; i < n; i++)
42         {
43             if(at[i] <= time)
44                 count++;
```

Code - Part 2 (Lines 45–80)

```
45          else
46              break;
47      }
48
49      // Sort available processes by burst time
50      if(count > 1)
51      {
52          for(i = over; i < over + count - 1; i++)
53          {
54              for(j = i + 1; j < over + count; j++)
55              {
56                  if(bt[i] > bt[j])
57                  {
58                      temp = at[i]; at[i] = at[j]; at[j] = temp;
59                      temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
60                      temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
61                  }
62              }
63          }
64      }
65
66      start = time;
67      time += bt[over];
68
69      printf("P[%d]\t%d\t%d\t%d\t%d\t%d\t%d\n",
70             pr[over],
71             at[over],
72             bt[over],
73             start,
74             time,
75             time - at[over] - bt[over],
76             time - at[over]);
77
78      sum_wait += time - at[over] - bt[over];
79      sum_turnaround += time - at[over];
80
```

Code - Part 3 (Lines 77–90)

```
77
78          sum_wait += time - at[over] - bt[over];
79          sum_turnaround += time - at[over];
80
81          over++;
82      }
83
84      avgwait = (float)sum_wait / n;
85      avgturn = (float)sum_turnaround / n;
86
87      printf("\nAverage waiting time: %.2f\n", avgwait);
88      printf("Average turnaround time: %.2f\n", avgturn);
89
90      return 0;
```

Output

```
Enter the number of processes:
3
Process 1:
Enter arrival time and burst time:
2
8
Process 2:
Enter arrival time and burst time:
2
3
Process 3:
Enter arrival time and burst time:
1
2

Process AT      BT      Start   End     WT      TAT
P[3]    1       2       0       2       -1      1
P[2]    2       3       2       5       0       3
P[1]    2       8       5       13      3       11

Average waiting time: 0.67
Average turnaround time: 5.00


...Program finished with exit code 0
Press ENTER to exit console.
```